```
print("KEERTESH U")
def get_neighbors(state):
    neighbors = []
    zero = state.index(0)

    row = zero // 3
    col = zero % 3

    moves = []

    if row > 0:
        moves.append(zero - 3)
    if row < 2:
        moves.append(zero + 3)
    if col > 0:
        moves.append(zero - 1)
    if col < 2:
        moves.append(zero + 1)      (variable) visited: set

    for move in moves:
        new_state = state[:]
        new_state[zero], new_state[move] = new_state[move], new_state[zero]
        neighbors.append(new_state)

    return neighbors


def dfs(start, goal):
    stack = [(start, 0)]
    visited = set()

    while stack:
        state, moves = stack.pop()

        if state == goal:
            return moves

        if tuple(state) in visited:
            continue

        visited.add(tuple(state))

        for next_state in get_neighbors(state):
            if tuple(next_state) not in visited:
                stack.append((next_state, moves + 1))

    return -1


start = list(map(int, input("Enter initial state: ").split()))

goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]
```

```
moves = dfs(start, goal)

if moves != -1:
    print("Number of moves:", moves)
else:
    print("No solution found.")
```

```
KEERTESH U
Enter initial state: 2 4 0 5 7 6 8 1 3
Number of moves: 113806
```

```
def get_neighbors(state):
    neighbors = []
    zero = state.index(0)

    row = zero // 3
    col = zero % 3

    moves = []
                                    (variable) visited: set
    if row > 0:
        moves.append(zero - 3)
    if row < 2:
        moves.append(zero + 3)
    if col > 0:
        moves.append(zero - 1)
    if col < 2:
        moves.append(zero + 1)

    for move in moves:
        new_state = state[:]
        new_state[zero], new_state[move] = new_state[move], new_state[zero]
        neighbors.append(new_state)

    return neighbors


def depth_limited_search(state, goal, depth, visited):
    if state == goal:
        return 0

    if depth == 0:
        return -1

    visited.add(tuple(state))

    for next_state in get_neighbors(state):
        if tuple(next_state) not in visited:
            result = depth_limited_search(
                next_state, goal, depth - 1, visited
            )

            if result != -1:
                return result + 1
```

```
    visited.remove(tuple(state))

    return -1


def iterative_deepening_search(start, goal, max_depth):
    for depth in range(max_depth + 1):
        visited = set()

        result = depth_limited_search(
            start, goal, depth, visited
        )

        if result != -1:
            return result

    return -1


print("8-PUZZLE USING ITERATIVE DEEPENING SEARCH")

print("\nEnter the initial state")
print("Use 0 for the blank space")

start = list(map(int, input("Enter 9 values: ").split()))

max_depth = int(input("Enter maximum depth: "))

if len(start) != 9 or set(start) != set(range(9)):
    print("\nInvalid input!")
    print("Enter exactly 9 numbers from 0 to 8.")

else:
    goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]

    moves = iterative_deepening_search(start, goal, max_depth)

    print("\nInitial State:", *start)
    print("Goal State:", *goal)
    print("Maximum Depth:", max_depth)

    if moves != -1:
        print("Solution found!")
        print("Number of moves:", moves)
    else:
        print("No solution found within the given depth.")
```

(variable) visited: set

```
8-PUZZLE USING ITERATIVE DEEPENING SEARCH

Enter the initial state
Use 0 for the blank space
Enter 9 values: 3 0 5 6 1 2 7 8 4
Enter maximum depth: 40

Initial State: 3 0 5 6 1 2 7 8 4
```

```
Goal State: 1 2 3 4 5 6 7 8 0
Maximum Depth: 40
Solution found!
Number of moves: 21
```

(variable) visited: set